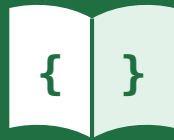

Modül 00 — Kurulum ve Modern Java (21 → 27)

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır?	2
Ödev 1 — Müşteri İndirimleri (Kolay)	2
Ödev 2 — Paralel Stok Sorgusu (Orta)	2
Ödev 3 — Sipariş Durum Makinesi (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır?

1. Başlangıç kodu `modules/00-setup-modern-java/exercise/` içindedir. TODO yorumlarını bulun.
2. Her ödevin testleri hazırdır ve siz çözene kadar **kırmızıdır**.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/00-setup-modern-java/exercise test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takıldığınızda önce ipuçlarına, sonra `modules/00-setup-modern-java/solution/` içindeki çözüme bakın.

★ İpucu

Tek bir ödevin testini çalıştırmak için: `-Dtest=Exercise1Test`

Ödev 1 — Müşteri İndirimleri (Kolay)

Hedef: Sealed bir tip üzerinde pattern matching ve when koşullarıyla karar vermek.

Kitapçının üç tür müşterisi var: Regular, Student(university) ve Member(years). Customer arayüzü sealed olduğu için başka müşteri türü olamaz.

Yapılacaklar (`exercise1.DiscountPolicy`):

- TODO 1a — `rateFor(customer)`: Regular için 0, Student için 10, Member için 5 döndürün. Üyelik 2 yıl ve üzeriyse 15, 5 yıl ve üzeriyse 20 olmalı.
- TODO 1b — `priceFor(customer, price)`: indirim fiyatı uygulayın ve sonucu 2 ondalığa yuvarlayın (HALF_UP).

İpuçları:

- Ders bölüm 3.2 ve 3.3: `case Member(int years) when years >= 5 -> ...`
- when koşulları yukarıdan aşağıya denenir. En özel durumu en üste yazın.
- default dalı eklemeyin. Derleyici kapsamlılığı sizin için kontrol eder.

Kabul kriterleri: `Exercise1Test` içindeki 4 testin hepsi geçer.

Tahmini süre: 20 dakika

Ödev 2 — Paralel Stok Sorgusu (Orta)

Hedef: Yavaş bir servise yapılan çok sayıda çağrıyı virtual thread'lerle paralel çalıştırmak.

`StockClient.stockOf(isbn)` her çağrıda bekleyen (bloklayan) bir uzak servis çağrısıdır. 200 kitabın stoğunu sırayla sormak 40 saniye sürer. Hedef, 3 saniyenin altına inmektir.

Yapılacaklar (exercise2.StockChecker):

- TODO 2a — Her ISBN için `client.stockOf(isbn)` çağrısını ayrı bir virtual thread’de çalıştırın.
- TODO 2b — Sonuçları giriş listesiyle **aynı sırada** döndürün.
- TODO 2c — `Future.get()`’in checked exception’larını `IllegalStateException`’a çevirin. `InterruptedException`’da interrupt bayrağını geri koymayı unutmayın.

İpuçları:

- Ders bölüm 3.5: `Executors.newVirtualThreadPerTaskExecutor()` ve `try-with-resources`.
- `LinkedHashMap` ekleme sırasını korur.
- Test, çağrılarının gerçekten virtual thread’de yapıldığını da kontrol eder.

Kabul kriterleri: `Exercise2Test` içindeki 2 testin ikisi de geçer.

Tahmini süre: 30 dakika

Ödev 3 — Sipariş Durum Makinesi (Zor)

Hedef: İki sealed hiyerarşiyi (durum × olay) iç içe record pattern’lerle birleştirerek kurallı bir durum makinesi yazmak.

Bir sipariş `New` → `Paid` → `Shipped` → `Delivered` akışını izler. `New` ve `Paid` durumundaki siparişler iptal (`Cancel`) edilebilir. Diğer tüm geçişler geçersizdir.

Yapılacaklar (exercise3.OrderStateMachine):

- TODO 3a — Geçerli geçişleri uygulayın:

Durum	Olay	Yeni durum
New	<code>Pay(amount)</code> , <code>amount > 0</code>	<code>Paid(amount)</code>
Paid	<code>Ship(trackingNumber)</code>	<code>Shipped(trackingNumber)</code>
Shipped	<code>Deliver</code>	<code>Delivered</code>
New veya Paid	<code>Cancel(reason)</code>	<code>Cancelled(reason)</code>

- TODO 3b — Diğer her durumda `IllegalStateException("Cannot <0lay> when <Durum>")` fırlatın, ör. `Cannot Ship when New`.

İpuçları:

- Durum ve olayı tek bir değerde birleştirin: `private record Transition(OrderState state, OrderEvent event)`.
- Sonra switch (`new Transition(state, event)`) ve iç içe pattern'ler: `case Transition(New _, Pay(BigDecimal amount)) when amount.signum() > 0 -> ...`
- Sınıf adı için: `event.getClass().getSimpleName()`.

Kabul kriterleri: Exercise3Test içindeki 4 testin hepsi geçer.

Tahmini süre: 45 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Ödev 2'yi structured concurrency ile yeniden yazın (ders bölüm 3.10). Kodunuzu `src/preview/java` altına koyun ve `-Ppreview` ile çalıştırın. Bir çağrı başarısız olduğunda diğerlerinin iptal edildiğini gösteren bir test yazın.